

Reverse Engineering & Executable Protection

ԲՈՎԱՆԴԱԿՈՒԹՅՈՒՆ

1. Ներածություն
 2. Reverse Engineering-ի գաղափարը
 3. Simple Executable Packer
 4. Packer ծրագրի կոդի բացատրություն
 5. Stub (Unpacker)
 6. Crackme ծրագիր
 7. Reverse Engineering Analysis
 8. Anti-Debugging
 9. Polymorphic / Metamorphic Obfuscation
 10. Օգտագործված ծրագրեր
 11. Գործնական արդյունքներ
 12. Եզրակացություն
-

1. ՆԵՐԱԾՈՒԹՅՈՒՆ

Reverse Engineering-ը ծրագրերի ուսումնասիրության և վերլուծության գործընթաց է, որի ընթացքում ուսումնասիրվում է executable ֆայլերի կառուցվածքը, machine code-ը և ծրագրի աշխատանքի տրամաբանությունը:

Այս աշխատանքի նպատակն է ուսումնասիրել executable protection-ի և reverse engineering-ի հիմնական մեթոդները:

Աշխատանքի ընթացքում ուսումնասիրվել են՝

- executable packer-ների աշխատանքը,
- XOR encryption,
- crackme ծրագրեր,

- anti-debugging մեխանիզմներ,
 - obfuscation տեխնոլոգիաներ:
-

2. REVERSE ENGINEERING-Ի ԳԱՂԱՓԱՐԸ

Reverse Engineering-ը թույլ է տալիս ուսումնասիրել executable ֆայլի ներքին կառուցվածքը՝ առանց source code ունենալու:

Օգտագործվում են հետևյալ գործիքները՝

- Ghidra
- x64dbg
- IDA Pro
- Hex Editor

Reverse Engineering-ի հիմնական նպատակներն են՝

- ծրագրի աշխատանքի հասկանալը,
 - vulnerability analysis,
 - malware analysis,
 - software protection ուսումնասիրելը:
-

3. SIMPLE EXECUTABLE PACKER

Packer-ի գաղափարը

Packer-ը executable ֆայլը encode կամ compress է անում և ստեղծում է նոր packed executable:

Runtime-ում packed executable-ը unpack է արվում և աշխատեցվում:

Այս աշխատանքում ստեղծվել է պարզեցված toy packer, որը օգտագործում է XOR encryption:

4. PACKER ԾՐԱԳԻՐ

PACKER SOURCE CODE

```
#include <iostream>
#include <fstream>
#include <vector>

using namespace std;

vector<char> read_file(const string& path) {
    ifstream file(path, ios::binary);
    return vector<char>((istreambuf_iterator<char>(file)),
                        istreambuf_iterator<char>());
}

void write_file(const string& path, const vector<char>& data) {
    ofstream file(path, ios::binary);
    file.write(data.data(), data.size());
}

void xor_encrypt(vector<char>& data, char key) {
    for (auto& b : data) {
        b ^= key;
    }
}

int main() {
    string input = "program.exe";
    string output = "packed.bin";
    char key = 0xAA;

    vector<char> data = read_file(input);

    xor_encrypt(data, key);

    write_file(output, data);

    cout << "Packed successfully!\n";
}
```

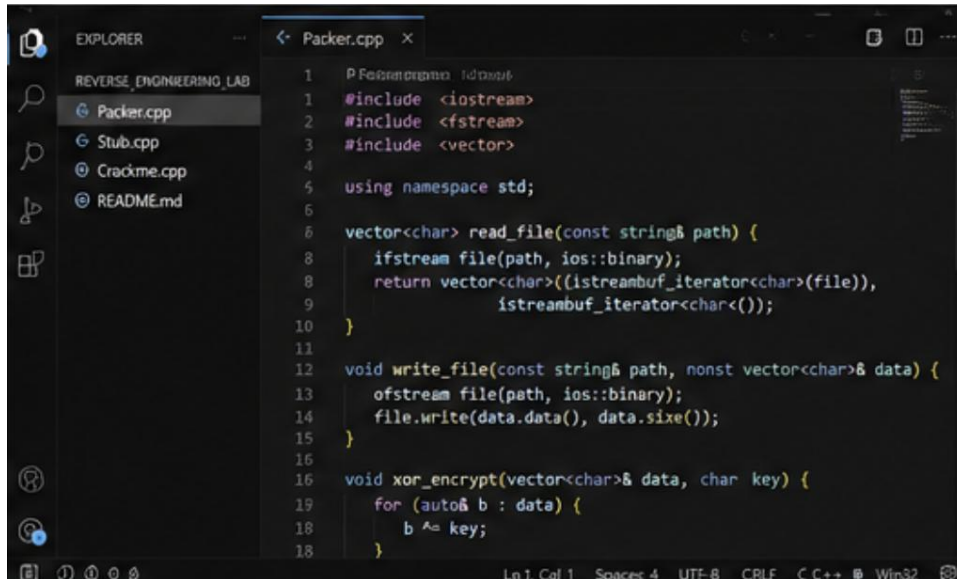
Կոդի բացատրություն

Ծրագիրը կատարում է հետևյալ քայլերը.

1. Բացում է executable ֆայլը
2. Կարդում է binary տվյալները
3. XOR-ով encode է անում
4. Ստեղծում է packed.bin ֆայլ

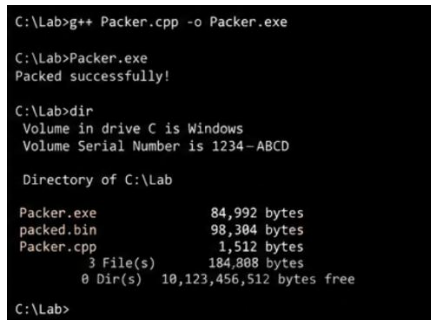
XOR encryption-ը ընտրված է ուսումնական նպատակներով:

VS Code / Code::Blocks պատուհան



```
1  Packer.cpp
2  #include <iostream>
3  #include <fstream>
4  #include <vector>
5
6  using namespace std;
7
8  vector<char> read_file(const string& path) {
9      ifstream file(path, ios::binary);
10     return vector<char>((istreambuf_iterator<char>(file)),
11                          istreambuf_iterator<char>());
12 }
13
14 void write_file(const string& path, const vector<char>& data) {
15     ofstream file(path, ios::binary);
16     file.write(data.data(), data.size());
17 }
18
19 void xor_encrypt(vector<char>& data, char key) {
20     for (auto& b : data) {
21         b ^= key;
22     }
23 }
```

Packer Compile Process

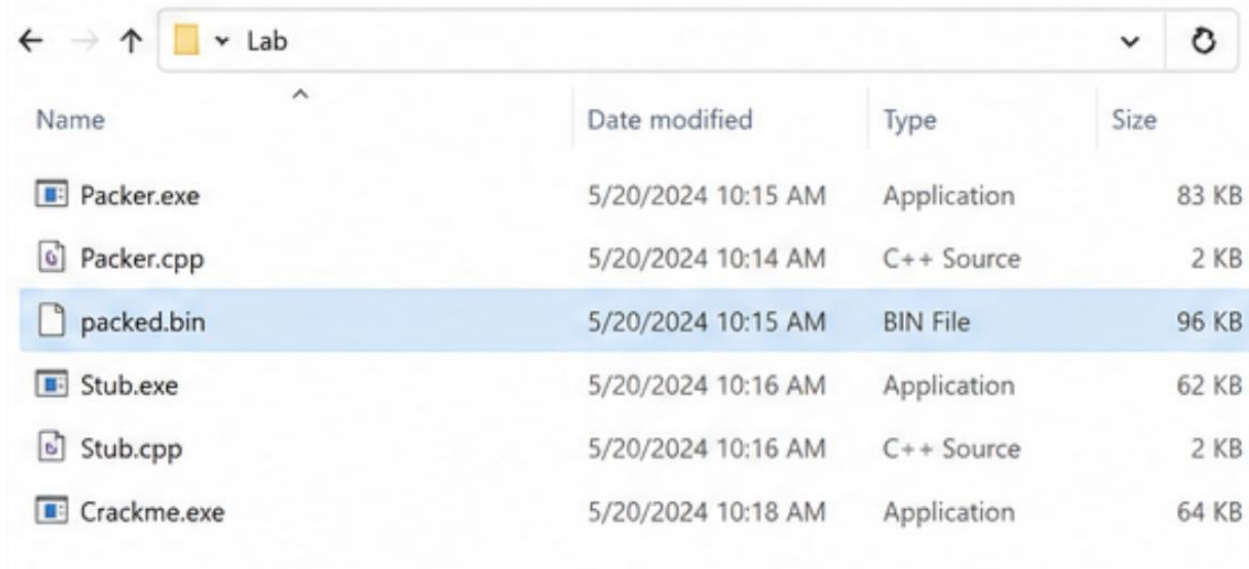


```
C:\Lab>g++ Packer.cpp -o Packer.exe
C:\Lab>Packer.exe
Packed successfully!
C:\Lab>dir
Volume in drive C is Windows
Volume Serial Number is 1234-ABCD

Directory of C:\Lab

Packer.exe             84,992 bytes
packed.bin             98,384 bytes
Packer.cpp              1,512 bytes
3 File(s)              184,888 bytes
0 Dir(s) 10,123,456,512 bytes free
C:\Lab>
```

packed.bin File



Name	Date modified	Type	Size
Packer.exe	5/20/2024 10:15 AM	Application	83 KB
Packer.cpp	5/20/2024 10:14 AM	C++ Source	2 KB
packed.bin	5/20/2024 10:15 AM	BIN File	96 KB
Stub.exe	5/20/2024 10:16 AM	Application	62 KB
Stub.cpp	5/20/2024 10:16 AM	C++ Source	2 KB
Crackme.exe	5/20/2024 10:18 AM	Application	64 KB

5. STUB (UNPACKER)

STUB SOURCE CODE

```
#include <iostream>
#include <fstream>
#include <vector>
#include <cstdlib>

using namespace std;

void xor_decrypt(vector<char>& data, char key) {
    for (auto& b : data) {
        b ^= key;
    }
}

int main() {
    ifstream file("packed.bin", ios::binary);

    vector<char> data((istreambuf_iterator<char>(file),
        istreambuf_iterator<char>()));
```

```
xor_decrypt(data, 0xAA);

ofstream out("unpacked.exe", ios::binary);
out.write(data.data(), data.size());

system("unpacked.exe");

return 0;
}
```

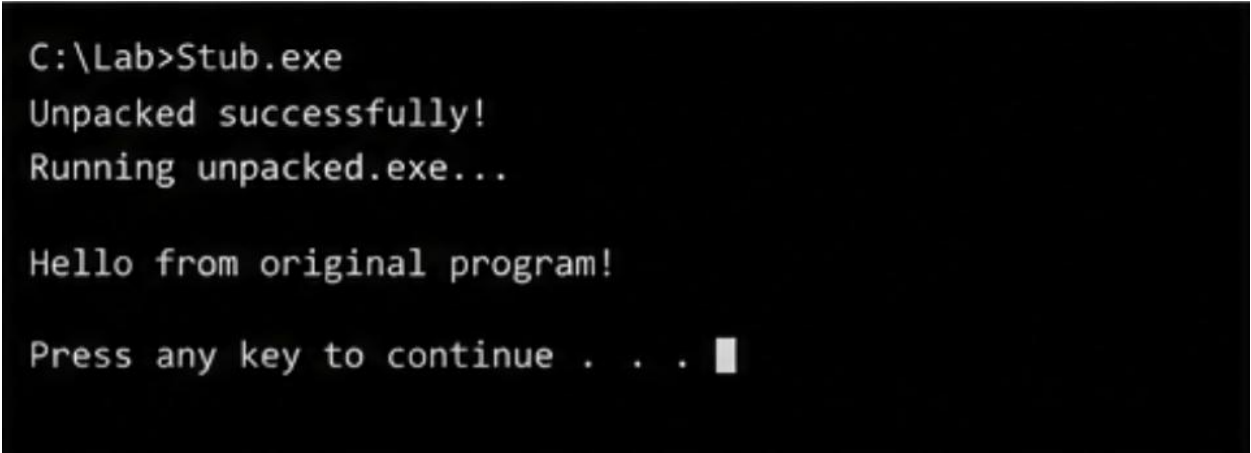
Stub-ի աշխատանքի նկարագրություն

Stub-ը runtime-ում կատարում է unpacking:

Այն՝

- բացում է packed.bin ֆայլը,
 - decode է անում տվյալները,
 - վերականգնում է executable-ը,
 - աշխատեցնում է այն:
-

Stub Execution



```
C:\Lab>Stub.exe
Unpacked successfully!
Running unpacked.exe...

Hello from original program!

Press any key to continue . . . █
```

6. CRACKME ԾՐԱԳԻՐ

CRACKME SOURCE CODE

```

#include <iostream>
#include <string>

using namespace std;

bool check(string input) {
    int key[] = {72, 29, 7, 0, 91};

    if (input.length() != 5)
        return false;

    for (int i = 0; i < 5; i++) {
        if ((input[i] ^ 0x55) != key[i])
            return false;
    }

    return true;
}

int main() {
    string pass;

    cout << "Enter password: ";
    cin >> pass;

    if (check(pass)) {
        cout << "Access Granted!\n";
    } else {
        cout << "Access Denied!\n";
    }

    return 0;
}

```

Crackme-ի աշխատանքի բացատրություն

Ծրագիրը password-ը ստուգում է XOR operation-ի միջոցով:

Verification logic-ը հետևյալն է.

$(input[i] \wedge 0x55) == key[i]$

Իրական password-ը encode է արված:

Crackme Execution (Wrong Password)

```
C:\Lab>Crackme.exe
Enter password: test

Access Denied!
Press any key to continue . . . █
```

Crackme Execution (Correct Password)

```
C:\Lab>Crackme.exe
Enter password: 1}y~q

Access Granted!
Press any key to continue . . . █
```

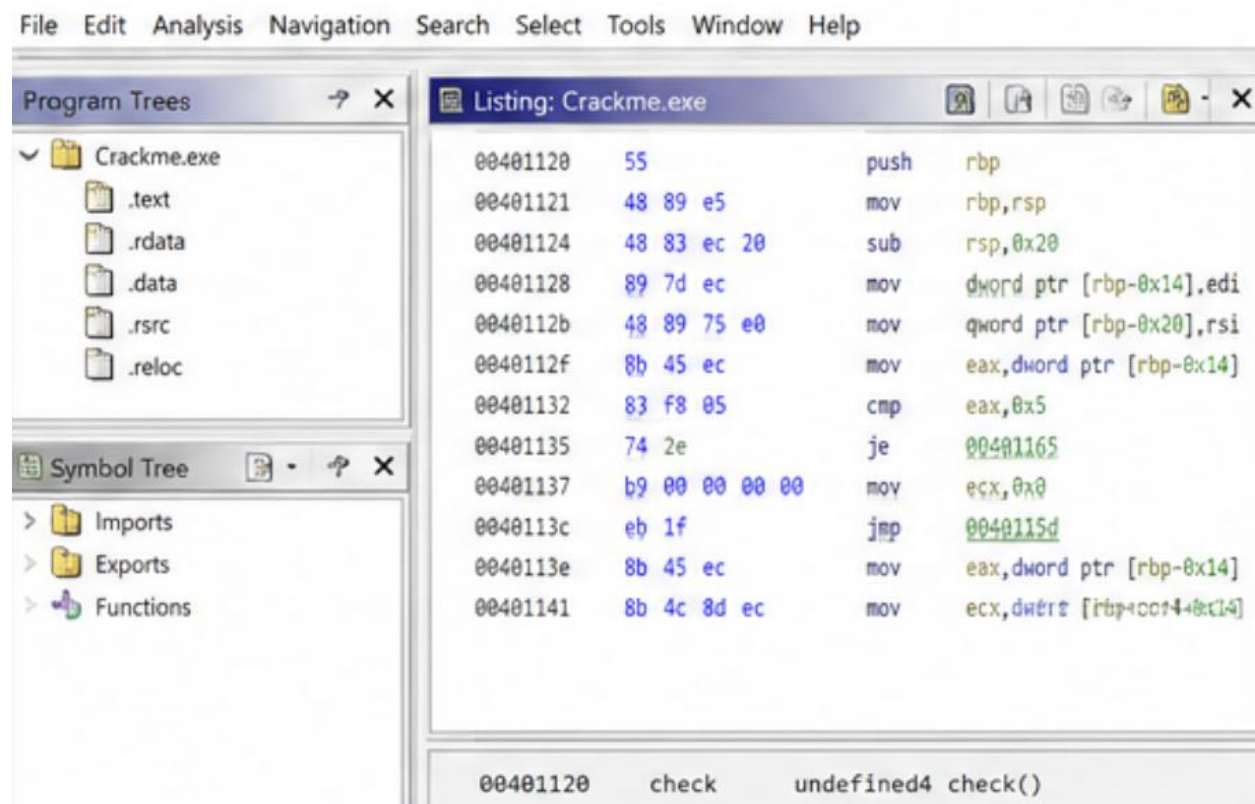
7. REVERSE ENGINEERING ANALYSIS

Reverse Engineering-ի ընթացքում executable ֆայլը ուսումնասիրվում է disassembler/debugger ծրագրերով:

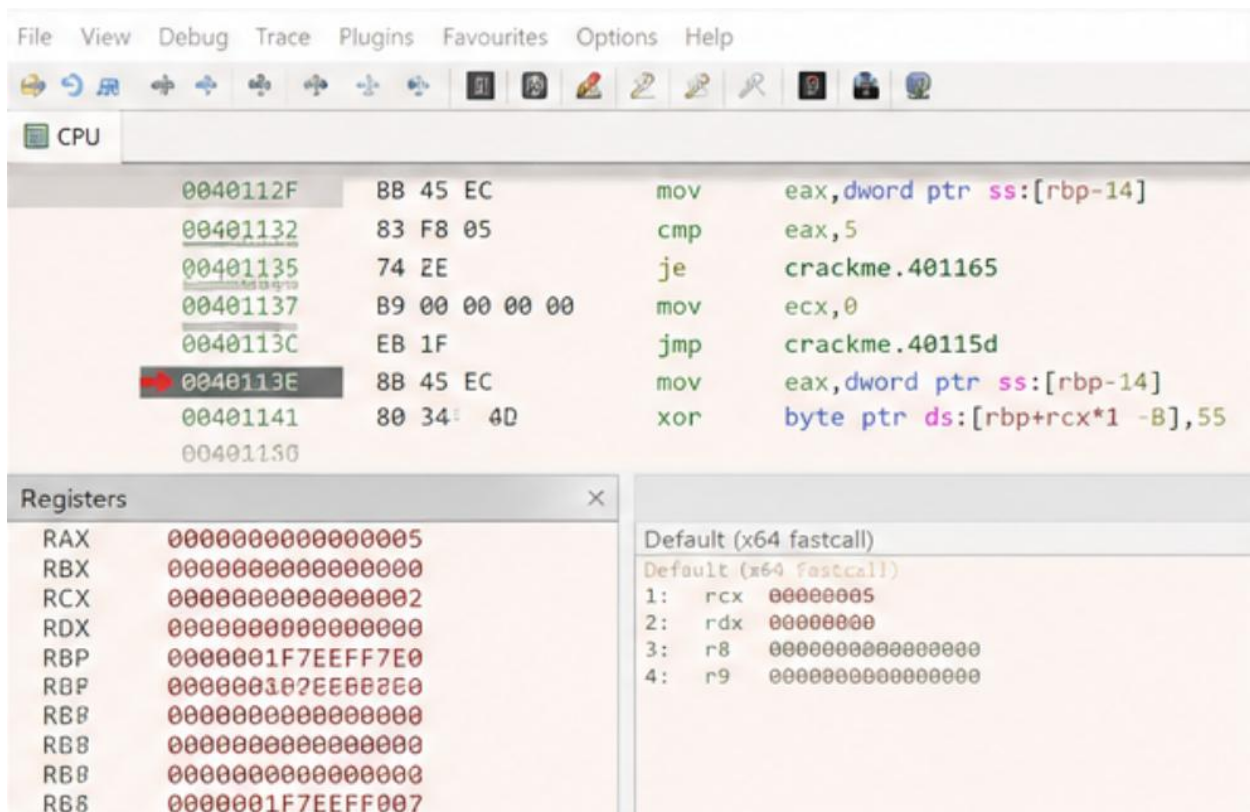
Ուսումնասիրվում են՝

- assembly instructions
- functions
- jumps
- password verification logic

Ghidra Disassembly Window



x64dbg Debugger Window



8. ANTI-DEBUGGING

Anti-debugging մեթոդները օգտագործվում են debugger-ների հայտնաբերման համար:

IsDebuggerPresent()

```
#include <windows.h>
```

```
if (IsDebuggerPresent()) {  
    exit(0);  
}
```

Այս ֆունկցիան ստուգում է debugger-ի առկայությունը:

ptrace()

Linux համակարգերում debugger detection-ի համար օգտագործվում է ptrace() ֆունկցիան:

9. POLYMORPHIC / METAMORPHIC OBFUSCATION

Polymorphic Obfuscation

Ծրագրի encoded տեսքը փոխվում է, սակայն logic-ը մնում է նույնը:

Metamorphic Obfuscation

Փոխվում է instruction structure-ը և program flow-ը:

Այս մեթոդը ավելի բարդ է:

Terminal / CMD Output

```
C:\Lab>g++ Packer.cpp -o Packer.exe
```

```
C:\Lab>Packer.exe  
Packed successfully!
```

```
C:\Lab>Stub.exe  
Unpacked successfully!  
Running unpacked.exe...
```

```
Hello from original program!
```

```
C:\Lab>Crackme.exe  
Enter password: test  
Access Denied!
```

```
C:\Lab>Crackme.exe  
Enter password: 1}y~q  
Access Granted!
```

```
C:\Lab>
```

10. ՕԳՏԱԳՈՐԾՎԱԾ ԾՐԱԳՐԵՐ

Աշխատանքի ընթացքում օգտագործվել են՝

- Visual Studio Code
 - Code::Blocks
 - GCC / MinGW
 - Ghidra
 - x64dbg
-

11. ԳՈՐԾՆԱԿԱՆ ԱՐԴՅՈՒՆՔՆԵՐ

Աշխատանքի արդյունքում ուսումնասիրվեցին՝

- executable file handling
 - XOR encryption
 - runtime unpacking
 - crackme structure
 - anti-debugging
 - reverse engineering analysis
-

12. ԵԶՐԱԿԱՑՈՒԹՅՈՒՆ

Այս աշխատանքի ընթացքում ուսումնասիրվեցին executable protection-ի և reverse engineering-ի հիմնական գաղափարները:

Ստեղծվեց պարզ executable packer և crackme ծրագիր:

Ուսումնասիրվեցին նաև anti-debugging և obfuscation մեթոդները: